

Lecture 9: Monads

COMP2221: Functional programming

Maximilien Gadouleau

`m.r.gadouleau@durham.ac.uk`

Monads

- In category theory, a monad is a functor with additional structure
- In Haskell, can consider it as an abstract datatype for actions (do notation as syntactic sugar for writing monadic expressions)
- Monads can be used to structure and compose computations
- Essentially, a standard programming interface for data and control structures

Monad type class

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
  (>>)  :: m a -> m b -> m b
  return :: a -> m a
```

- `return`:
 - wrap a value in a context - resulting in the monadic value `m a`
 - `return = pure` !
 - Note: not like `return` in imperative programming languages - does not end function execution
- Bind operator `>>=`:
 - Compose two actions, passing any value produced by the first as an argument to the second
 - Definition contains instance-dependent implementation of additional actions
- Bind operator `>>` without value passing

- Monads have to fulfill monad laws to behave properly
 - Left identity: `return a >>= f <=> f a`
- Wrapping a value in a context and binding it to a function is the same as applying the function to the extracted value
- Right identity: `m >>= return <=> m`
- Taking a monadic value and binding it to return leaves the monadic value unchanged
- Associativity:
$$(m \gg= (\backslash x \rightarrow g\ x)) \gg= (\backslash y \rightarrow h\ y) \leq=> m \gg= (\backslash x \rightarrow g\ x \gg= (\backslash y \rightarrow h\ y))$$

Maybe monad

- **Maybe** monad represents computations which can fail by not returning a value

```
data Maybe a = Just a | Nothing

instance Monad Maybe where
  -- (>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
  Nothing >>= f = Nothing
  Just x >>= f  = f x

  -- return :: a -> Maybe a
  return x = Just x
```

Toy example: safe division

- Suppose we introduce expressions made up of integers and the division sign, e.g.

```
div 5 (div 29 11)
```

- We would like to evaluate those expressions (e.g. above we should get 2). We can do it as follows

```
data Expr = Val Int | Div Expr Expr
```

```
eval :: Expr -> Int
```

```
eval (Val n) = n
```

```
eval (Div x y) = div (eval x) (eval y)
```

- But this has the problem of potentially dividing by zero, so we should work with `Maybe Int`'s instead.

Toy example: safe division

- The evaluation can then be performed as follows:

```
safediv :: Int -> Int -> Maybe Int
safediv _ 0 = Nothing
safediv n p = Just (n `div` p)
```

```
safeeval :: Expr -> Maybe Int
safeeval (Val n)      = Just n
safeeval (Div x y)    = safeeval x >=> \n ->
                        safeeval y >=> \p ->
                        safediv n p
```

- `safeeval Div x y` first evaluates `x` and calls its result `n`, then evaluates `y` and calls its result `p`, and finally combines the two results by applying `safediv`

Toy example: safe division

- Haskell provides a more elegant (and more familiar) notation for

```
mx1 >>= \n1 ->  
mx2 >>= \n2 ->  
f n1 n2
```

namely

```
do n1 <- mx1  
   n2 <- mx2  
   f n1 n2
```

- (We have already seen the `do` command, for the `main` program. In fact, this can be used for any monad.)
- So, we can simplify `safeeval` to

```
safeeval :: Expr -> Maybe Int  
safeeval (Val n)    = Just n  
safeeval (Div x y)  = do  
    n <- safeeval x  
    p <- safeeval y  
    safediv n p
```

The `List` monad

- Unsurprisingly, the list type is another monad, where

```
instance Monad [] where
  -- (>>=) :: [a] -> (a -> [b]) -> [b]
  xs >>= f = [y | x <- xs, y <- f x]
```

That is, `xs >>= f` applies the function `f` to each of the results in the list `xs`, collecting all the values in a list.

- For instance, we can write the `map` application in monadic style:

```
Prelude> range :: Int -> [Int]; range x = [0..x]
Prelude> [1,2] >>= range
[0,1,0,1,2]
```

Example for composition with bind: function chaining

The `IO` type is another monad. This allows us to perform sequences of actions with input/output strings of characters

```
stringToNum :: String -> IO Int
stringToNum s = return (read s)

inc :: Int -> IO Int
inc x = return (x + 1)

main :: IO ()
main = getLine >>= stringToNum >>= inc >>= print
```

Equivalently, with `do` notation as syntactic sugar:

```
main :: IO ()
main = do
    input <- getLine
    num <- stringToNum input
    result <- inc num
    print result
```

- **Maybe**: provides context to model failure
- **IO**: represents IO actions $(\gg=) :: \text{IO } a \rightarrow (a \rightarrow \text{IO } b) \rightarrow \text{IO } b$, e.g., to allow waiting for user input; `getLine` does not return a **String** but is rather an IO action which resolves as a string on evaluation.
- **List**: binding means joining together a set of calculations for each value in the list - $(\gg=) :: [a] \rightarrow (a \rightarrow [b]) \rightarrow [b]$
- And many others: **Either**, **MonadState**, **MonadReader**, **MonadWriter**, ...

Aside: Probabilistic Functional Programming

Let us discuss a monad for dealing with probabilities and randomness in Haskell: <https://web.engr.oregonstate.edu/~erwig/pfp/>